University of BRISTOL

Bristol Cyber Security Group

Systems Design(Basics)

Fundamentals of System Security
(COMSM0122)

bristol.ac.uk

Dr Inah Omoronyia
MVB 2.39A
Email: Inah.Omoronyia@bristol.ac.uk

Learning Outcome

University of BRISTOL
Bristol Cyber Security Group

- Describe the properties of a well-designed system
- Understand how to design a system that satisfies scalability requirements, is resilient and mitigates against denial-of-service attacks
- Describe different load-balancing algorithms
- Compare generate and compare different system design configurations using different evaluation metrics.
- Describe how to secure the architecture of a system design.

Data centers worldwide: https://map.datacente.rs

Failure Rates in Google Data Centers

In each cluster's first year, it's typical that 1,000 individual machine failures will occur; thousands of hard drive failures will occur; one power distribution unit will fail, bringing down 500 to 1,000 machines for about 6 hours; 20 racks will fail, each time causing 40 to 80 machines to vanish from the network; 5 racks will "go wonky," with half their network packets missing in action; and the cluster will have to be rewired once, affecting 5 percent of the machines at any given moment over a 2-day span, Dean said. And there's about a 50 percent chance that the cluster will overheat, taking down most of the servers in less than 5 minutes and taking 1 to 2 days to recover.

Google fellow Jeff Dean

What is Systems Design?

University of BRISTOL
Bristol Cyber Security Group

The process of defining the **architecture**, **components**, **modules**, **interfaces**, and **data** for a system to satisfy specified requirements.

- Involves defining and transforming user requirements into an architecture that best satisfies them.

- By considering factors such as:
  - Scalability
  - Reliability
  - Performance
  - Consistency
  - Secure

Example Scenario

University of BRISTOL
Bristol Cyber Security Group

You have written a popular algorithm on your computer, and people are willing to pay you so that they can use your code.

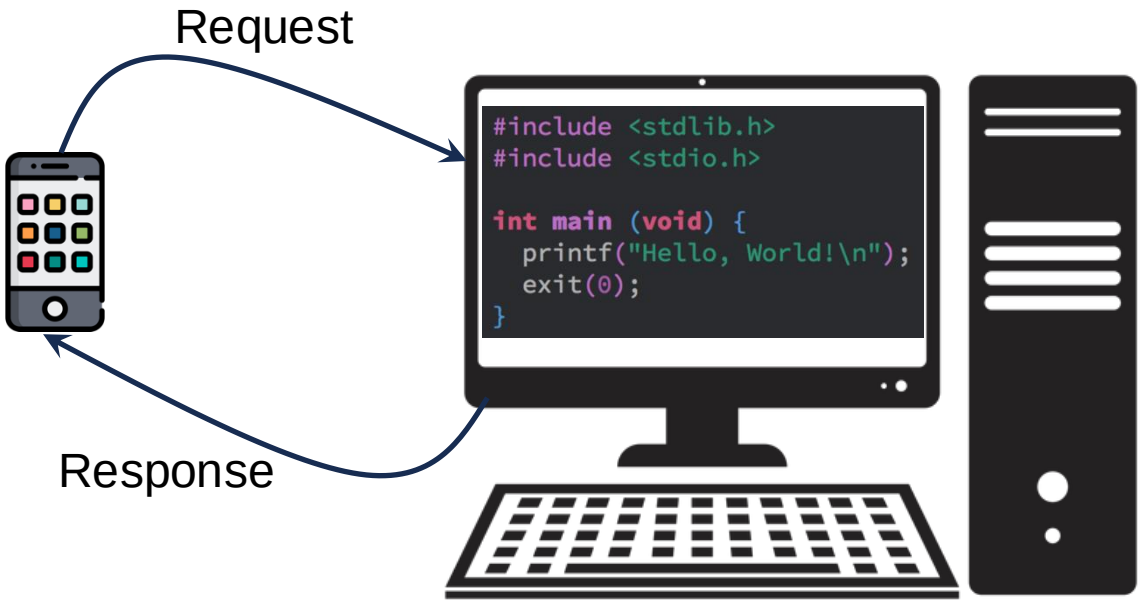
- But you cannot go around giving your computer to whoever wants to use your code…
- So your plan is to expose your code using some protocol which is going to be running on the internet, and by using an API (Application Programming Interface).

Example Scenario

University of BRISTOL
Bristol Cyber Security Group

You have written a popular algorithm on your computer, and people are willing to pay you so that they can use your code.

- When a person makes a request to your algorithm, your code runs and returns an output as a response instead of storing that in the file or database.
- How would you setup a computer to offer the service?



Option 1: Self Hosting

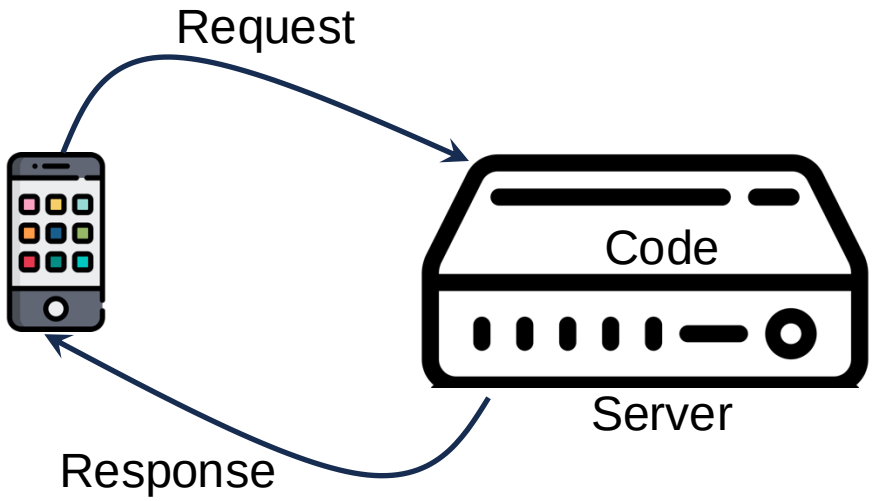
University of BRISTOL
Bristol Cyber Security Group

- With your desktop
- May require you to connect your desktop to a database.
- You also need to configure endpoints that people would connect to
- Well, since people are paying you, you cannot afford to have your service go down – so you need to consider what happens if there is a power loss, and that someone does not steal your desktop, etc.

Option 2: Cloud Hosting

University of BRISTOL
Bristol Cyber Security Group

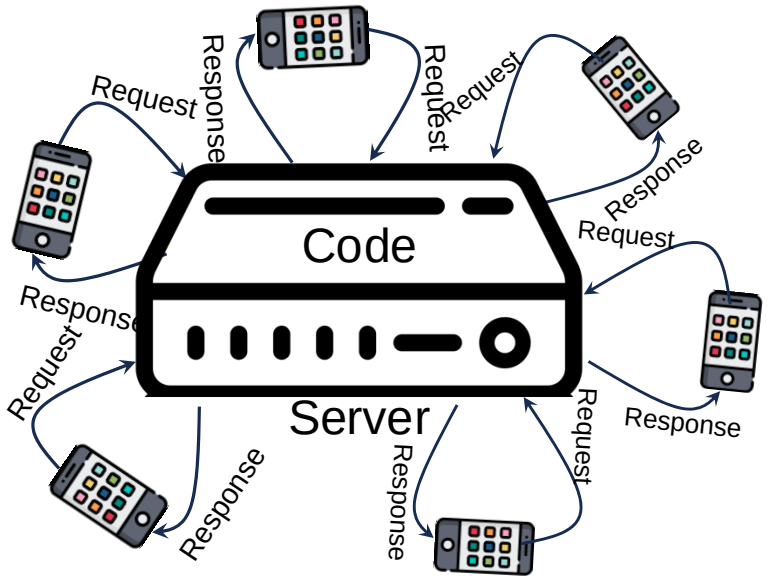
- You pay for computational power from a cloud provider
- The cloud provider gives you a desktop (a remote computer) to deploy your algorithm.
- You login remotely to the computer to upload and run your algorithm
- The advantage is that the configuration, settings and reliability are now the responsibilities of the cloud provider.
- You can now focus on the business requirements



Scalability Requirements

University of BRISTOL
Bristol Cyber Security Group

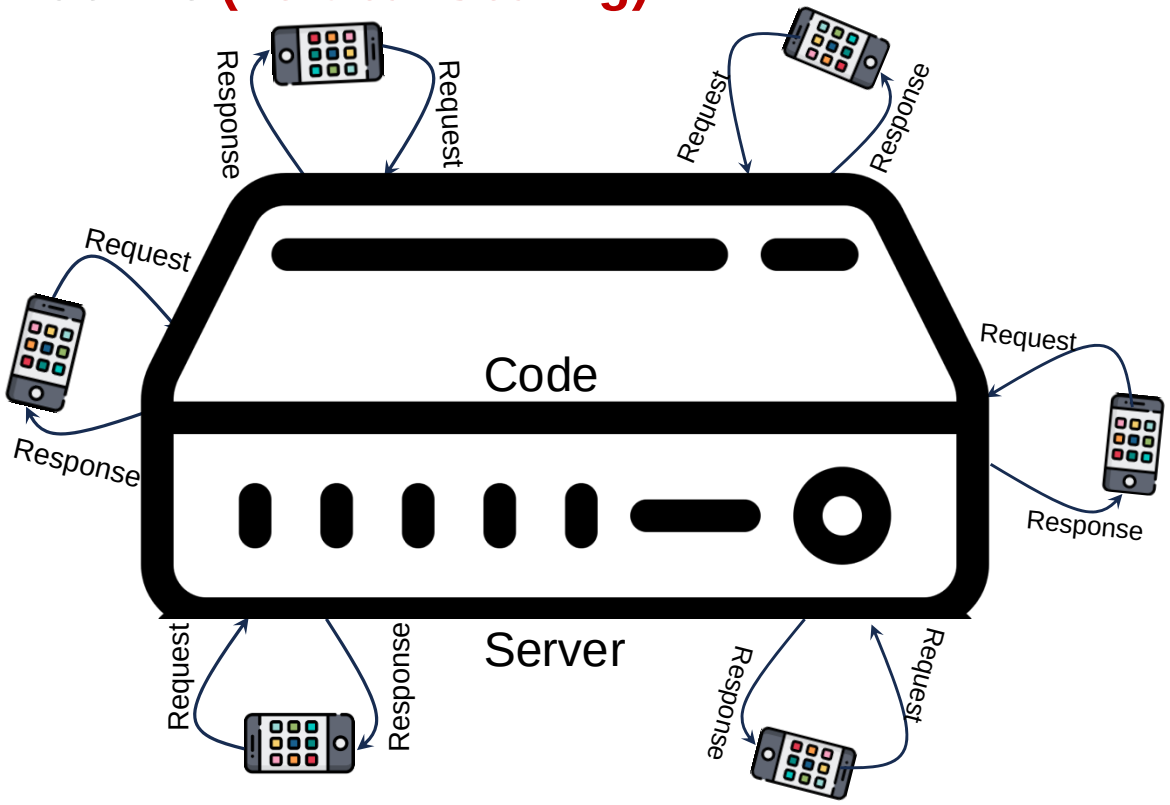
- A lot of people are now using your algorithm, and it's getting to a point where the code is running on a machine that is not able to handle the connection of all users.
- What do you do?



Scalability Requirements

University of BRISTOL
Bristol Cyber Security Group

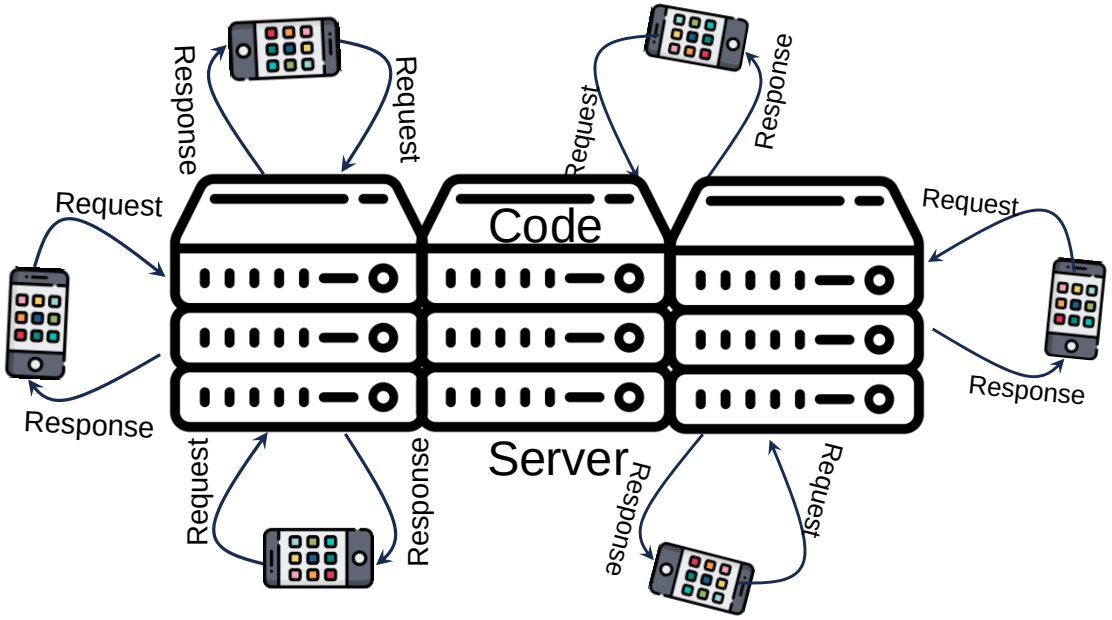
- A lot of people are now using your algorithm, and it's getting to a point where the code is running on a machine that is not able to handle the connection of all users.
- Soln 1: Buy a bigger machine **(Vertical Scaling)**



Scalability Requirements

University of BRISTOL
Bristol Cyber Security Group

- A lot of people are now using your algorithm, and it's getting to a point where the code is running on a machine that is not able to handle the connection of all users.
- Soln 2: Buy a more machines **(Horizontal Scaling)**

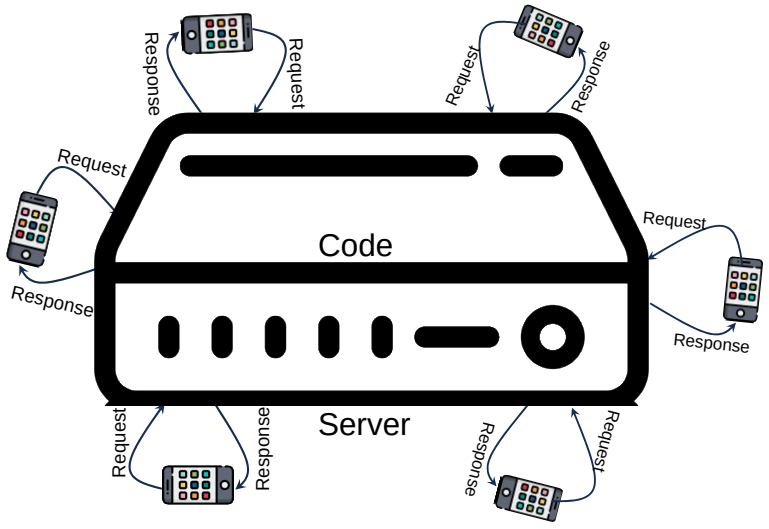


Scalability

University of BRISTOL
Bristol Cyber Security Group

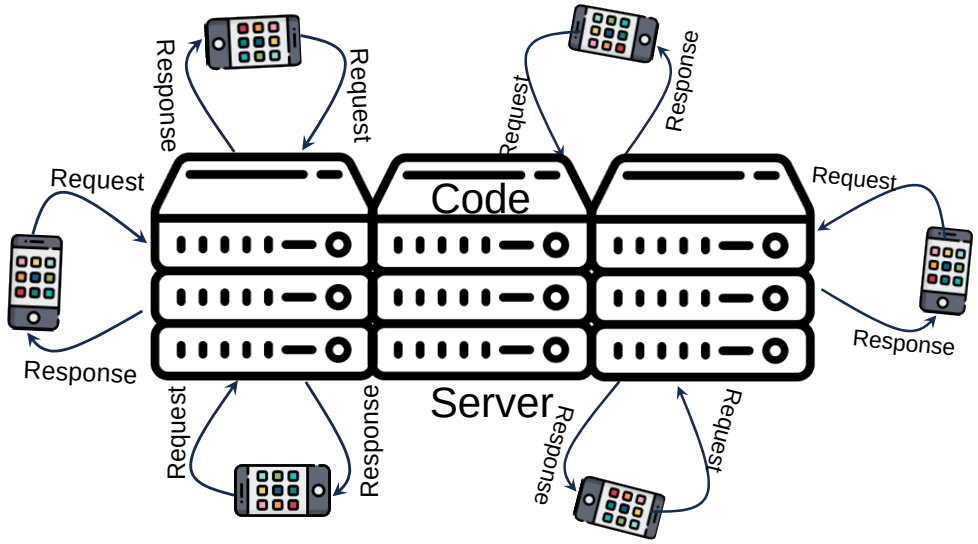
- Scalability is the ability to handle more requests at peak times by using a bigger machine or using more machines, **and to handle fewer requests at off-peak times by using a smaller machine or using fewer machines**.

Vertical Scaling



The computer is larger and therefore processes requests faster.

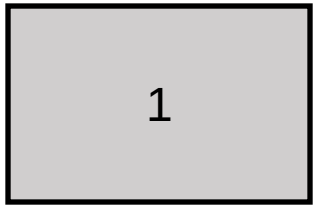
Horizontal Scaling



Requests are randomly distributed amongst machines

Horizontal vs Vertical Scaling

University of BRISTOL
Bristol Cyber Security Group

Horizontal		**Vertical**	
1.		1.	
2.	Load balancing required	2.	N/A
3.	You redirect requests when one fails.	3.	Single point of failure
4.	Network calls (RPC)	4.	Inter-process communication
5.	Data inconsistency	5.	Data consistent
6.	Scales well as users increase	6.	Hardware limit

Practical Considerations

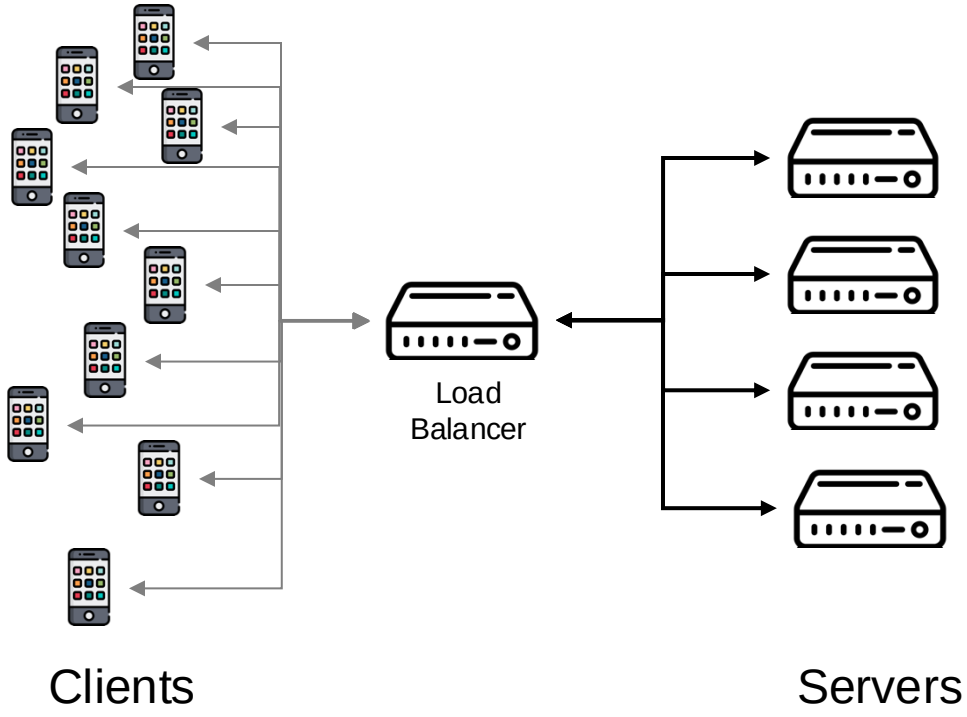
University of BRISTOL
Bristol Cyber Security Group

- What scaling approach do you think is used in the real world?

Load Balancing

University of BRISTOL
Bristol Cyber Security Group

- The aim is to distribute incoming requests (traffic) evenly across multiple servers to ensure high availability, reliability, and performance.

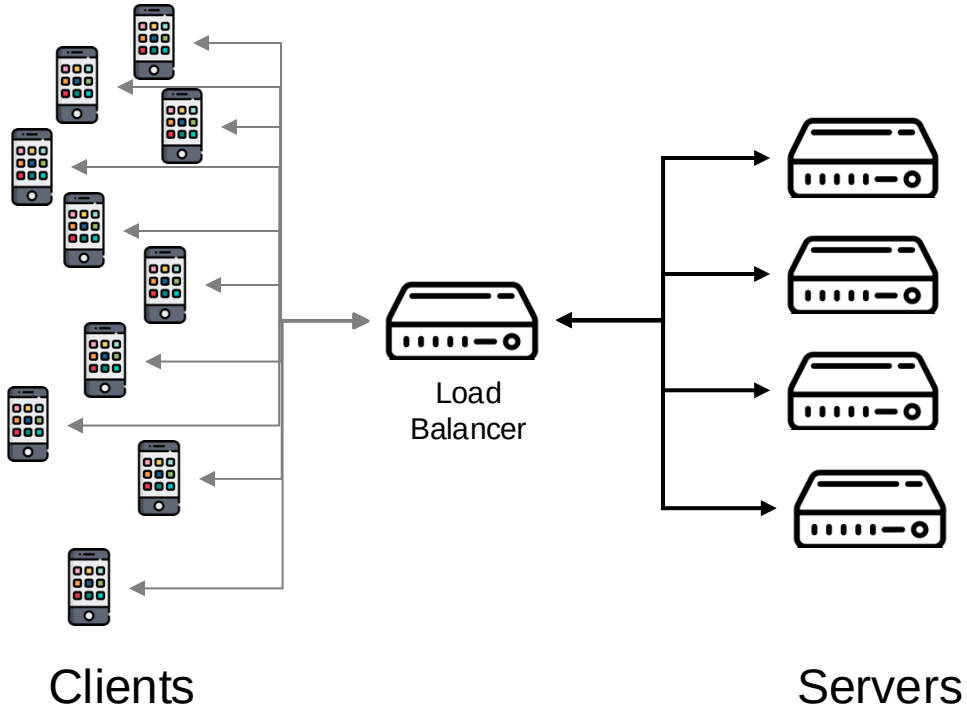


Load Balancing

University of BRISTOL
Bristol Cyber Security Group

- How do you balance load across N servers?

Ensure efficient utilization of available resources, improve overall system performance, and maintain high availability and reliability.



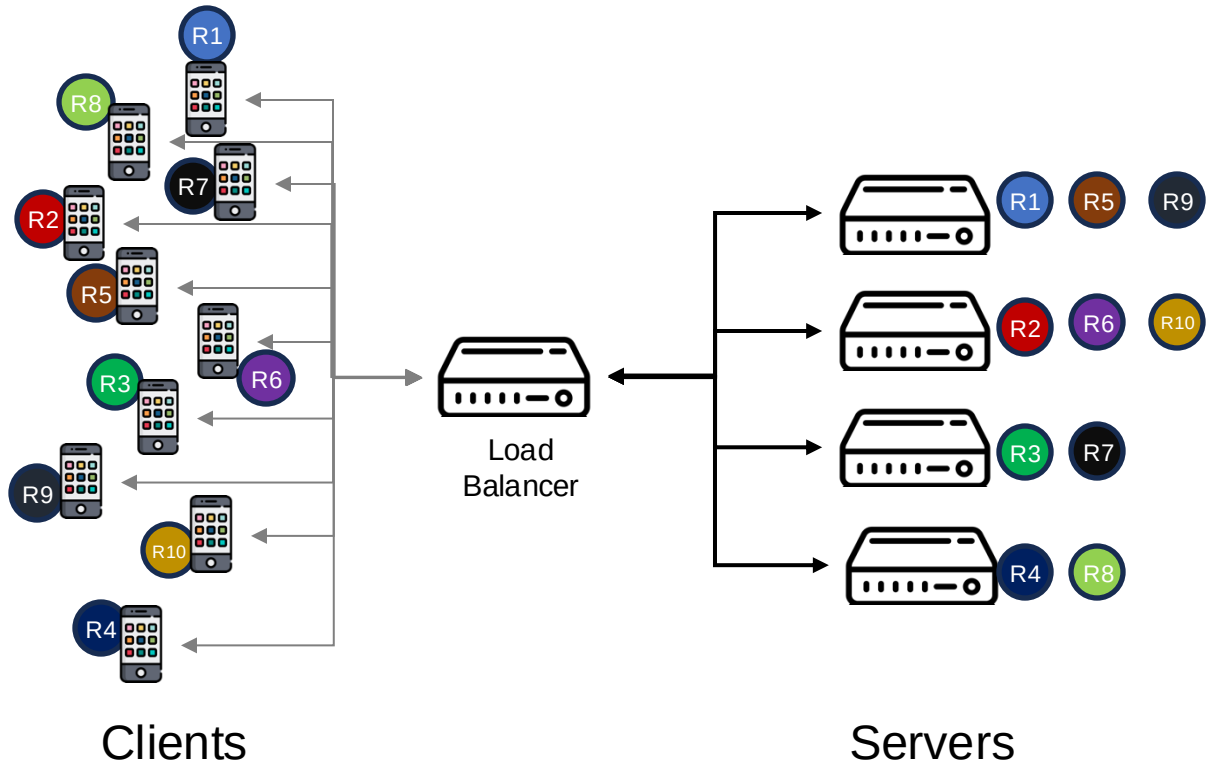
Load Balancing Algorithms

University of BRISTOL
Bristol Cyber Security Group

Round Robin

- Distributes incoming requests to servers in a cyclic order, and each server gets a turn in a fixed order.

- It's most effective when client requests are predictable and the servers have similar processing capabilities and resources.

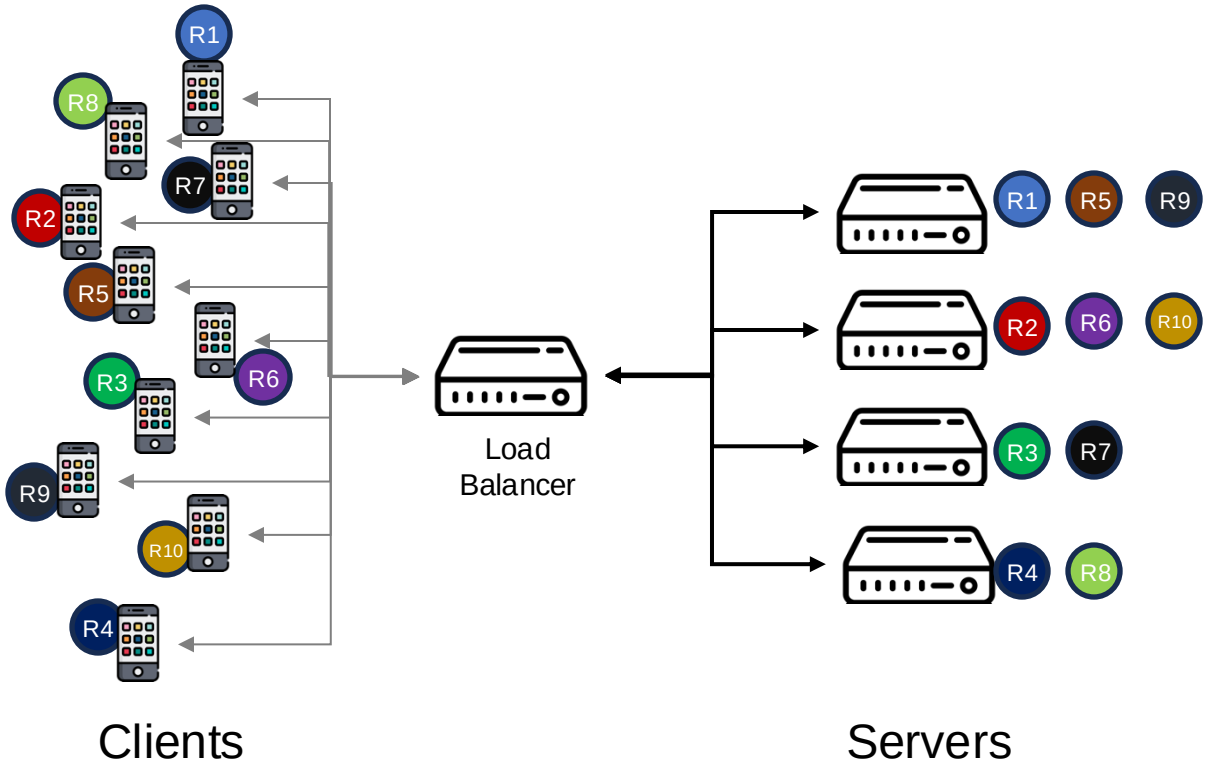


Load Balancing Algorithms

University of BRISTOL
Bristol Cyber Security Group

Round Robin

- Pros:
  - Easy to implement and understand
  - Works well when servers have similar capacities
- Cons:
  - No load awareness
  - No session affinity (subsequent requests from the same client may be directed to different servers)

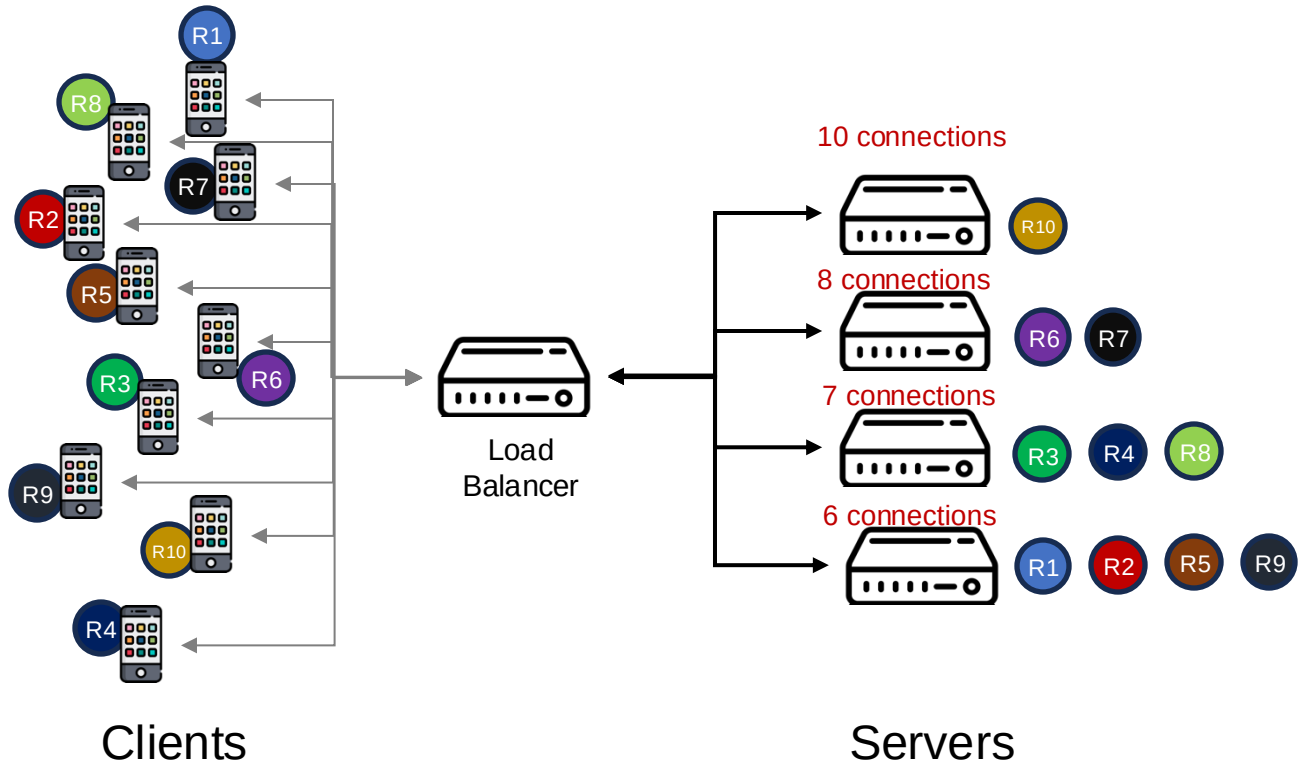


Load Balancing Algorithms

University of BRISTOL
Bristol Cyber Security Group

Least Connections

- A dynamic load balancing technique that assigns incoming requests to the server with the fewest active connections at the time of the request.

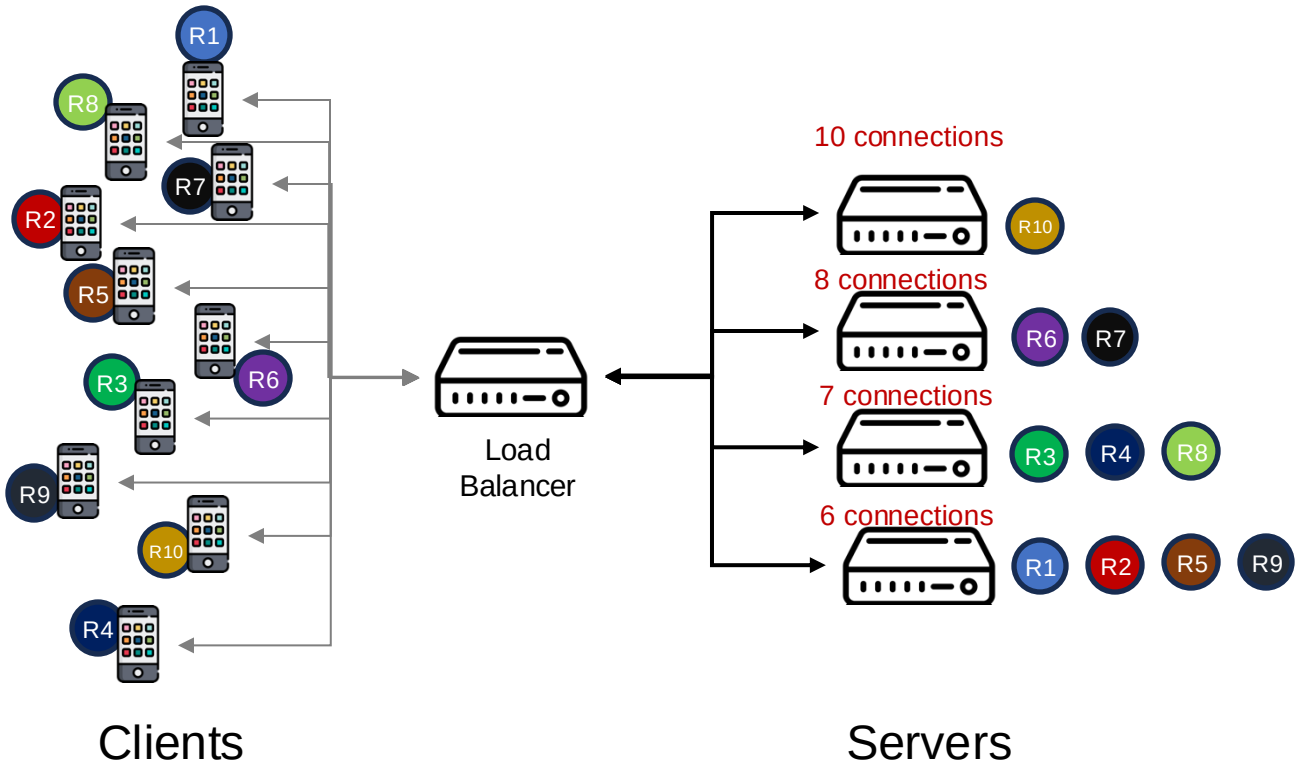


Load Balancing Algorithms

University of BRISTOL
Bristol Cyber Security Group

Least Connections

- Pros:
  - Load awareness
  - Dynamic distribution
  - Efficient in a heterogeneous environment
- Cons
  - Higher complexity
  - State maintenance



Quiz

University of BRISTOL
Bristol Cyber Security Group

- Which of the following are advantages of load balancing
  a) Enable horizontal scaling
  b) Dynamic scaling
  c) Abstraction
  d) Enable vertical scaling
  e) Throughput
  f) Redundancy
  g) Continuous deployment
  h) a and b
  i) a, b, c and f
  j) a, b, c, e, f and g
  k) a, b, c, d, e, f, and g

Problem

University of BRISTOL
Bristol Cyber Security Group

Imagine a system expected to serve **stateful** *R* requests per day across *N* different servers.

In addition, consider that the configuration of these servers can change over time:

Examples:

- Any server can be shut down
- A new server can be added to the system

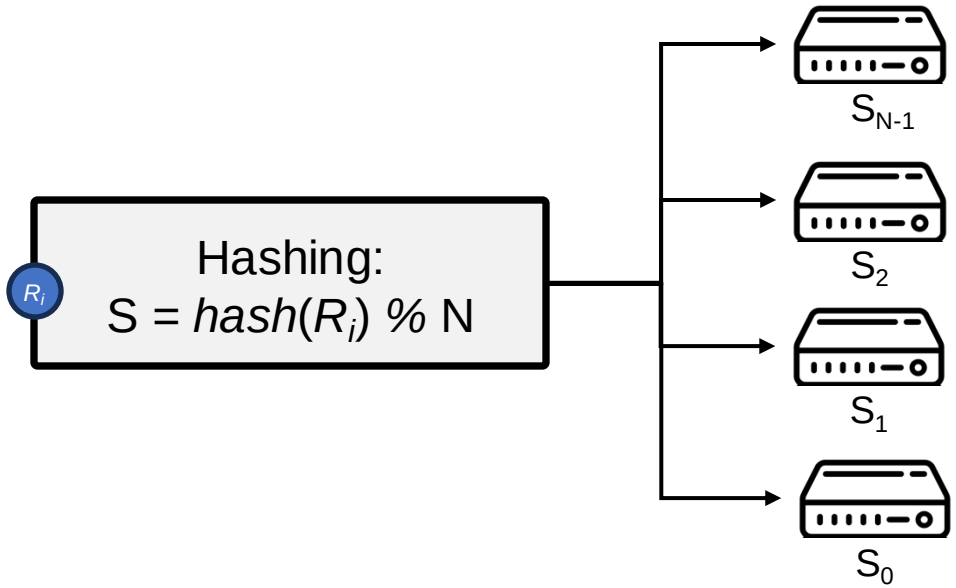
Given these potential changes, the aim is to design a system that can rapidly respond to user requests when configurations change.

Hashing Implementation

University of BRISTOL
Bristol Cyber Security Group

Distribute requests across different servers based on a **hash function**

- You get a uniformly random **request ID** R_i for each request.
- Number of servers 0 – *N-1*
- Use the hash value of request ID to determine which server responds to a request
- Stateful session is achieved since the same hash value is returned for request with the same ID.

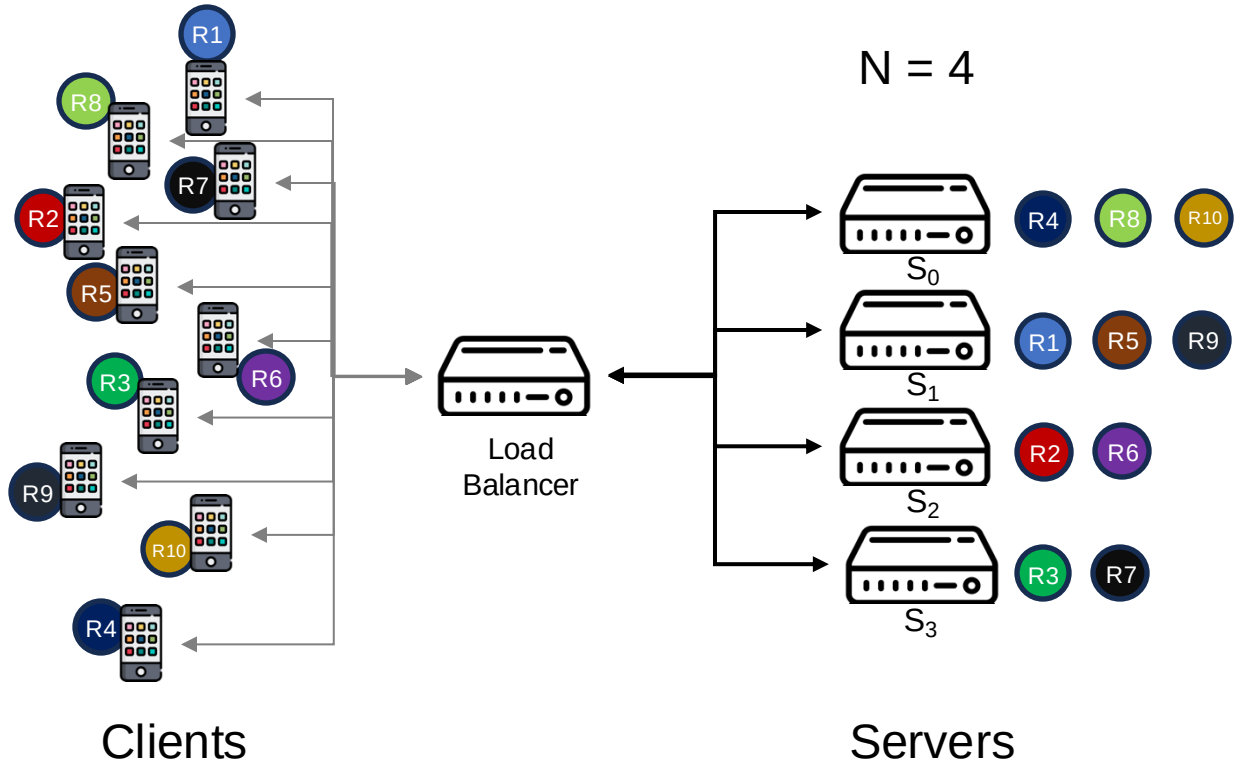


Hashing Implementation

University of BRISTOL
Bristol Cyber Security Group

Distribute requests across different servers based on a **hash function**

- *H= hash(R1)* = 1361, H % 4 = 1
- *H= hash(R2)* = 1362, H % 4 = 2
- *H= hash(R3)* = 1363, H % 4 = 3
- *H= hash(R4)* = 1364, H % 4 = 0
- *H= hash(R5)* = 1365, H % 4 = 1
- *H= hash(R6)* = 1366, H % 4 = 2
- *H= hash(R7)* = 1367, H % 4 = 3
- *H= hash(R8)* = 1368, H % 4 = 0
- *H= hash(R9)* = 1369, H % 4 = 1
- *H= hash(R10)* = 21824, H % 4 = 0

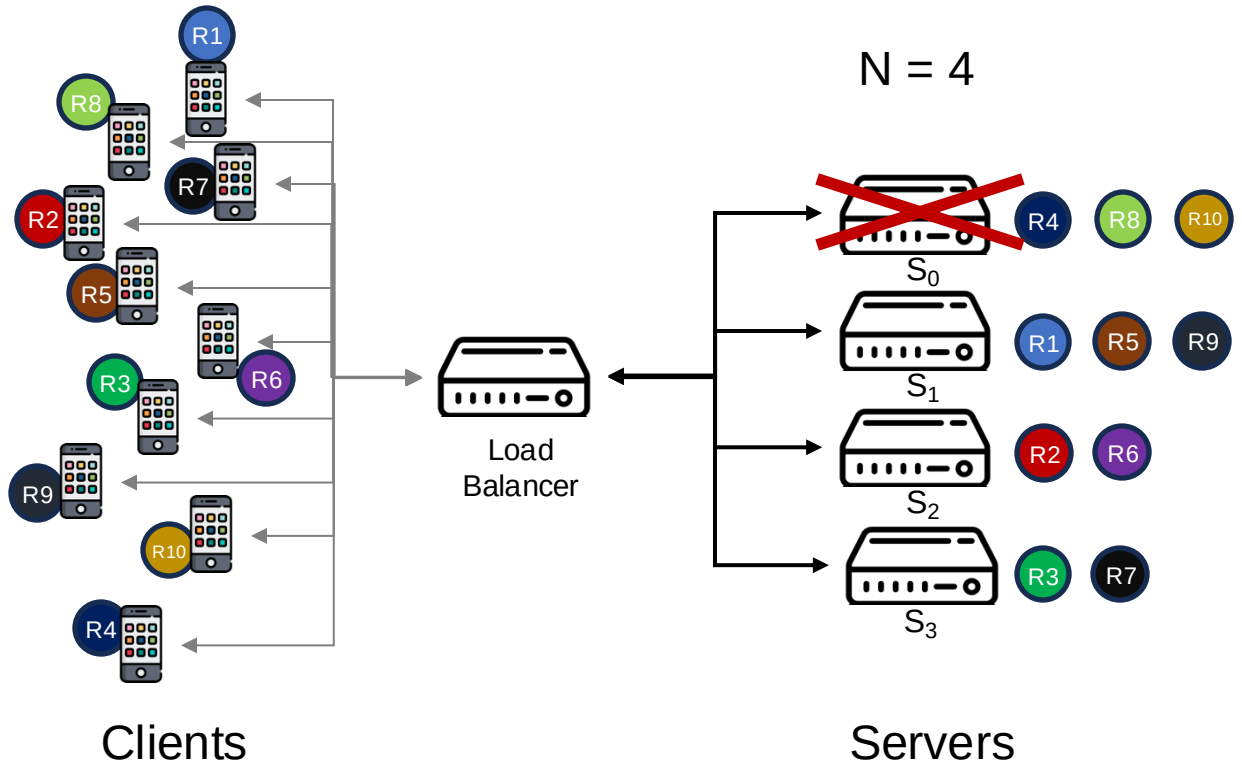


Hashing Implementation

University of BRISTOL
Bristol Cyber Security Group

Rehashing Problem

- When a server is shut down, its client's sessions are cut off, necessitating the redistribution of requests across the remaining servers
- Adding a new server requires modifying the hash function to account for the new server.
- *All sessions will likely need to be rehashed into different servers (in this example using* H *%* 3)

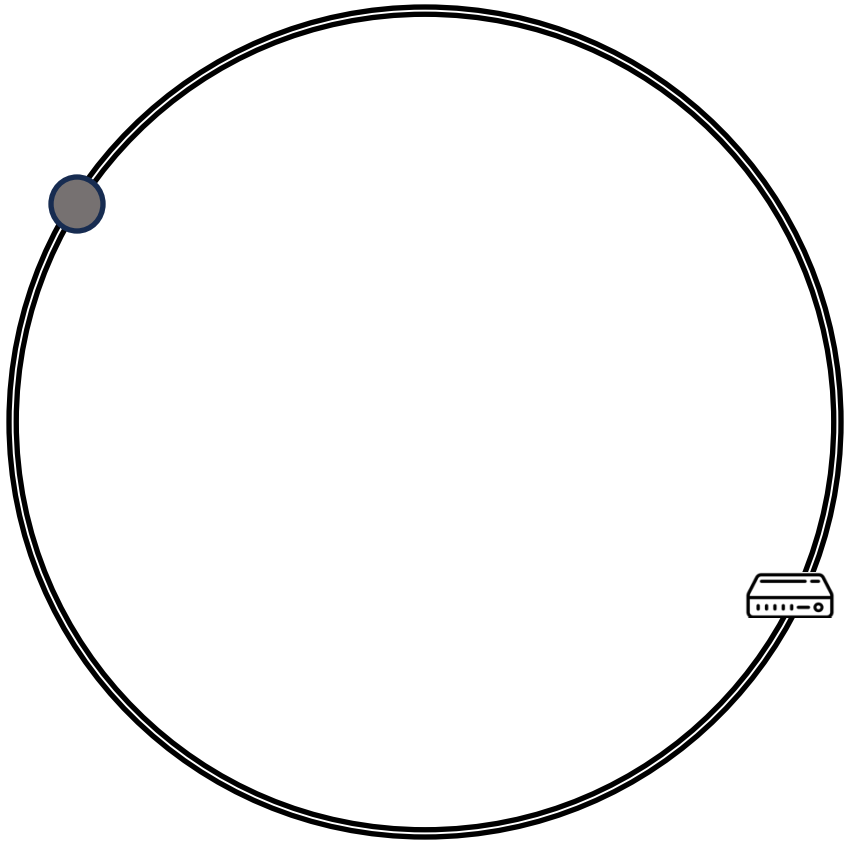


Consistent Hashing

University of BRISTOL
Bristol Cyber Security Group

A resilient alternative to the basic hashing system

- It represents the resource requestors (requests) and the server in a virtual ring structure known as a hashring
- The ring is considered to have an infinite number of points, and the server nodes can be placed at random locations on this ring (using a hash function).
- Requests IDs are also placed on the same ring using the same hash function.

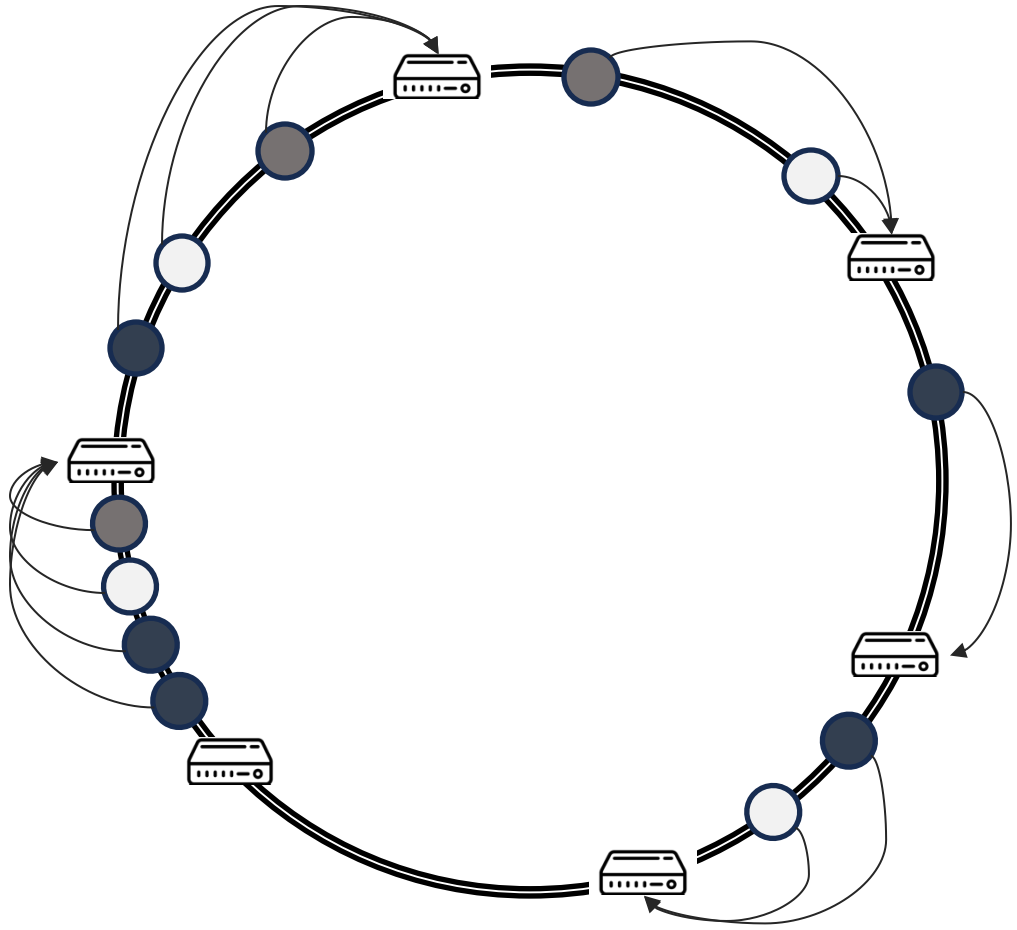


Consistent Hashing

University of BRISTOL
Bristol Cyber Security Group

How is the decision about which request will be served by which server node made?

- Assume the ring is ordered so that clockwise traversal of the ring corresponds to increasing order of location addresses, each request can be served by the server node that first appears in this clockwise traversal.



Consistent Hashing

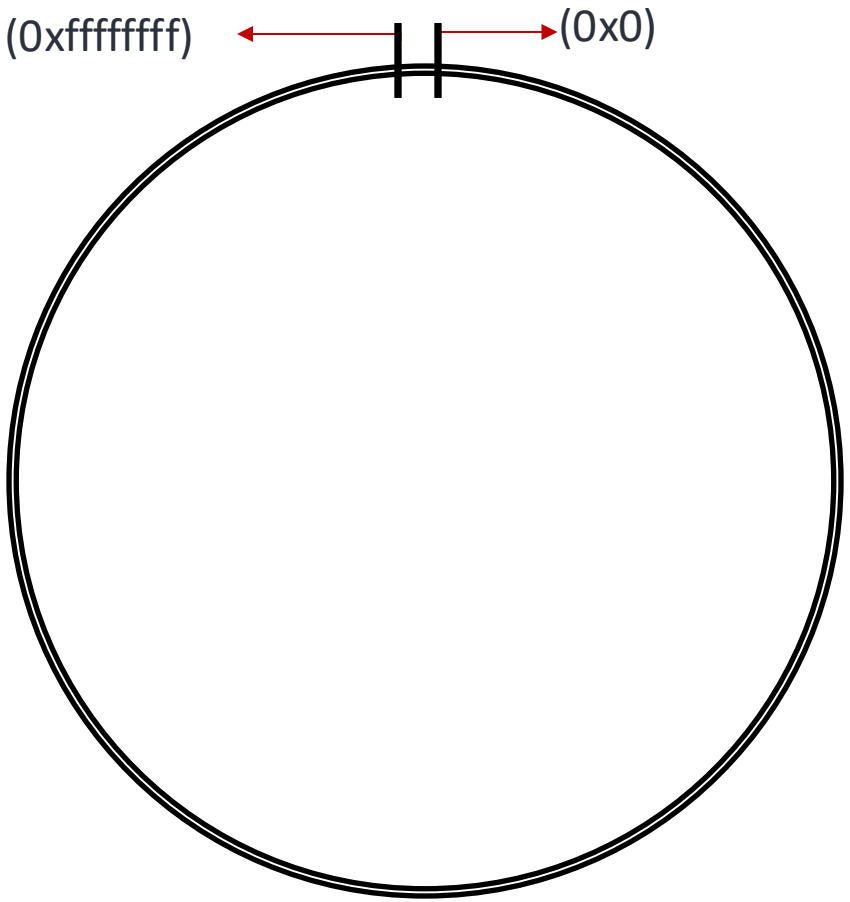
University of BRISTOL
Bristol Cyber Security Group

- The minimum possible hash value (zero) would correspond to an angle of zero.

- The maximum possible hash value would correspond to an angle of 2π radians (or 360 degrees)
- All other hash values would linearly fit somewhere in between.

For example:

- Assuming 32-bit hashes, then place ring where the numbers 0x0, 0x1, 0x2… are placed consecutively up to 0xffffffff, then returns through the circle to be followed by 0x0.



Consistent Hashing

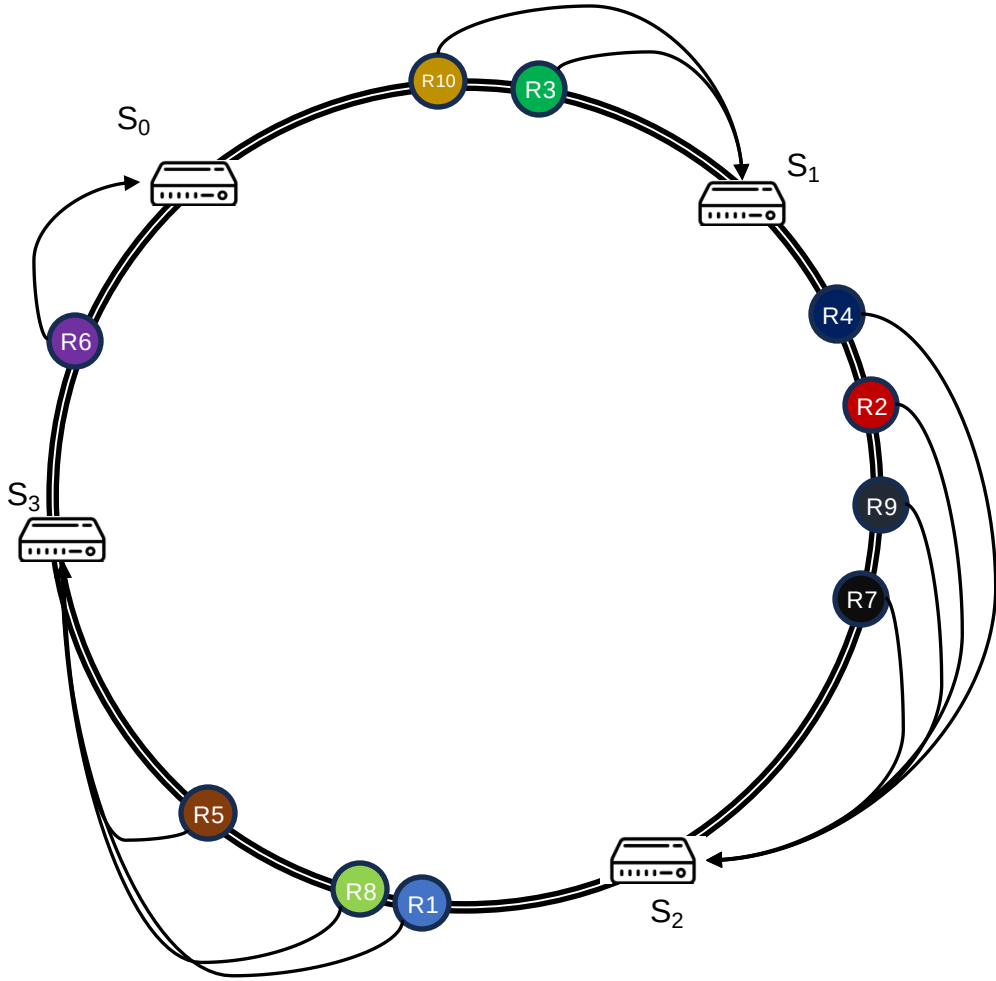
University of BRISTOL
Bristol Cyber Security Group

Distribute requests across different servers based on a **hash function**

```
let hash = crypto.createHash('md5').update(`S${j}`).digest('hex');
```

- *hash*(*S0*) = fa4373030709881197fd6070eaecb61f
- *hash*(*S1*) = 67f6274e0ac0bd892f9b1ec09a2253fc
- *hash*(*S2*) = b9eeaf6a16ca49f37df57620aed91b62
- *hash*(*S3*) = e2ab7c65b21ed8cc1c3b642b5e36429e

- *hash*(*R1*) = cda522d4353b166cc2dee84673307b4e
- *hash*(*R2*) = 8c6d22ff6f63fc6711cfa315cb80b314
- *hash*(*R3*) = 5c108ce0fe89d0632cfce75f650b36c2
- *hash*(*R4*) = 8717ce4dfdc86a4b576d9e983ab9fb29
- *hash*(*R5*) = e1b8054c9cdd622c9eeecfe71f26054a
- *hash*(*R6*) = f38ee76ce06ccd13940482aced1c7391
- *hash*(*R7*) = b9d7e51eba2c6d155e7fd3013338c38e
- *hash*(*R8*) = cfff813d86d447fa2a9c858650ebbb90
- *hash*(*R9*) = 8d28dad91e1ffada38203b9e5f9af86d
- *hash*(*R10*) = 33df7d5541bc0941336b1513815da7a5



Consistent Hashing

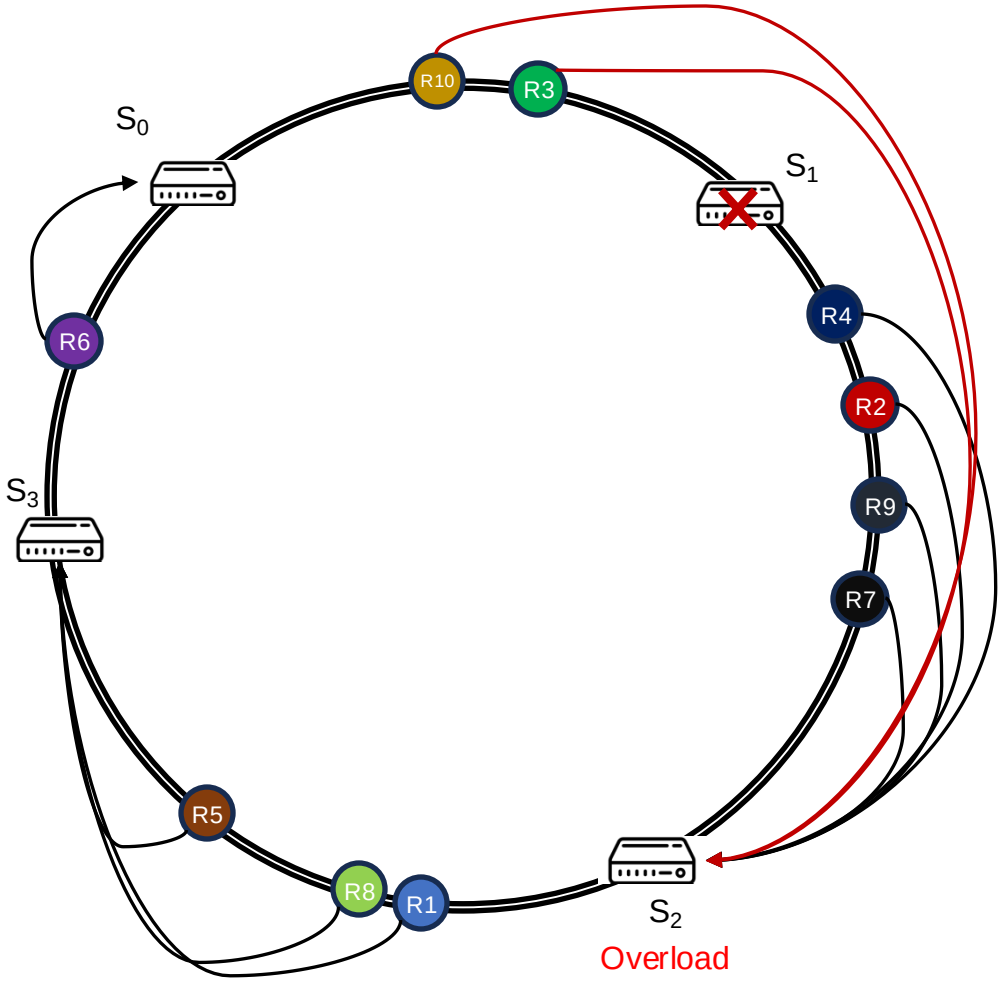
University of BRISTOL
Bristol Cyber Security Group

Addition or removal of a server node

```
let hash = crypto.createHash('md5').update(`S${j}`).digest('hex');
```

- *hash*(*S0*) = fa4373030709881197fd6070eaecb61f
- *hash*(*S1*) = 67f6274e0ac0bd892f9b1ec09a2253fc
- *hash*(*S2*) = b9eeaf6a16ca49f37df57620aed91b62
- *hash*(*S3*) = e2ab7c65b21ed8cc1c3b642b5e36429e

- *hash*(*R1*) = cda522d4353b166cc2dee84673307b4e
- *hash*(*R2*) = 8c6d22ff6f63fc6711cfa315cb80b314
- *hash*(*R3*) = 5c108ce0fe89d0632cfce75f650b36c2
- *hash*(*R4*) = 8717ce4dfdc86a4b576d9e983ab9fb29
- *hash*(*R5*) = e1b8054c9cdd622c9eeecfe71f26054a
- *hash*(*R6*) = f38ee76ce06ccd13940482aced1c7391
- *hash*(*R7*) = b9d7e51eba2c6d155e7fd3013338c38e
- *hash*(*R8*) = cfff813d86d447fa2a9c858650ebbb90
- *hash*(*R9*) = 8d28dad91e1ffada38203b9e5f9af86d
- *hash*(*R10*) = 33df7d5541bc0941336b1513815da7a5

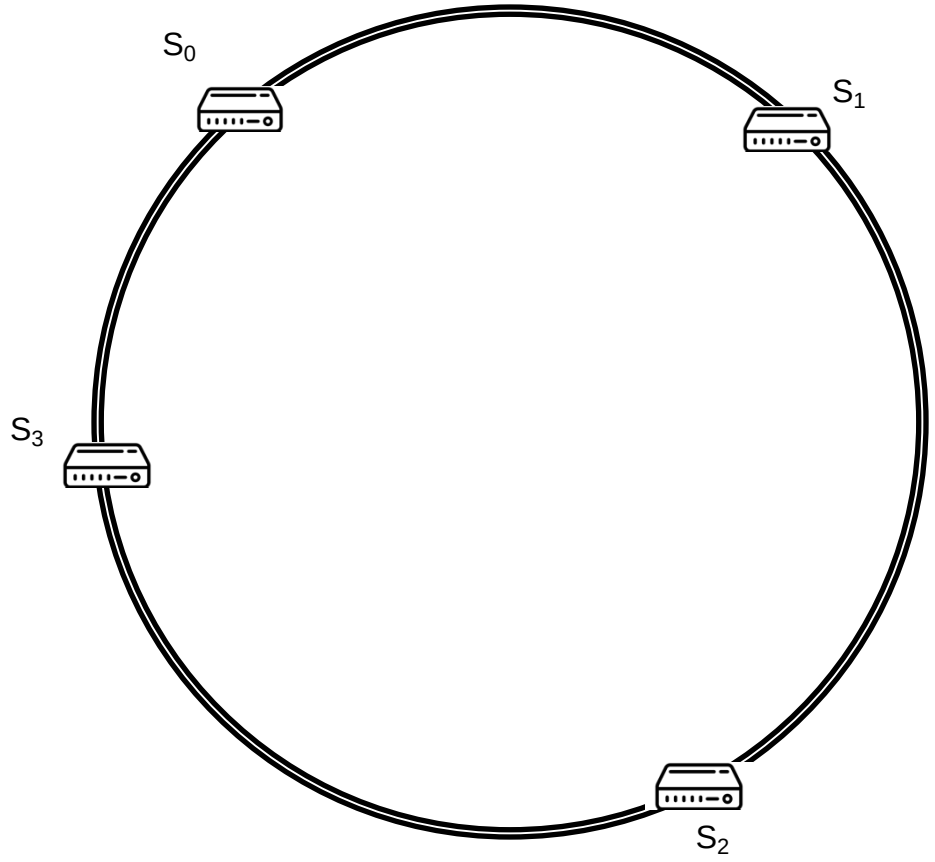


Consistent Hashing

University of BRISTOL
Bristol Cyber Security Group

Create replicas of servers (virtual servers)

- To evenly distribute the load among servers when a server is added or removed, it creates a fixed number of replicas (known as virtual nodes) of each server and distributed it along the circle.
- Instead of server labels S0, S1, S2, and S3, we will have **S00 S01…S09, S10 S11…S19, S20 S21…S29, S30 S31…S39 and S40 S41…S49**
- All keys which are mapped to replicas **Sij** are stored on server **Si**.



Consistent Hashing

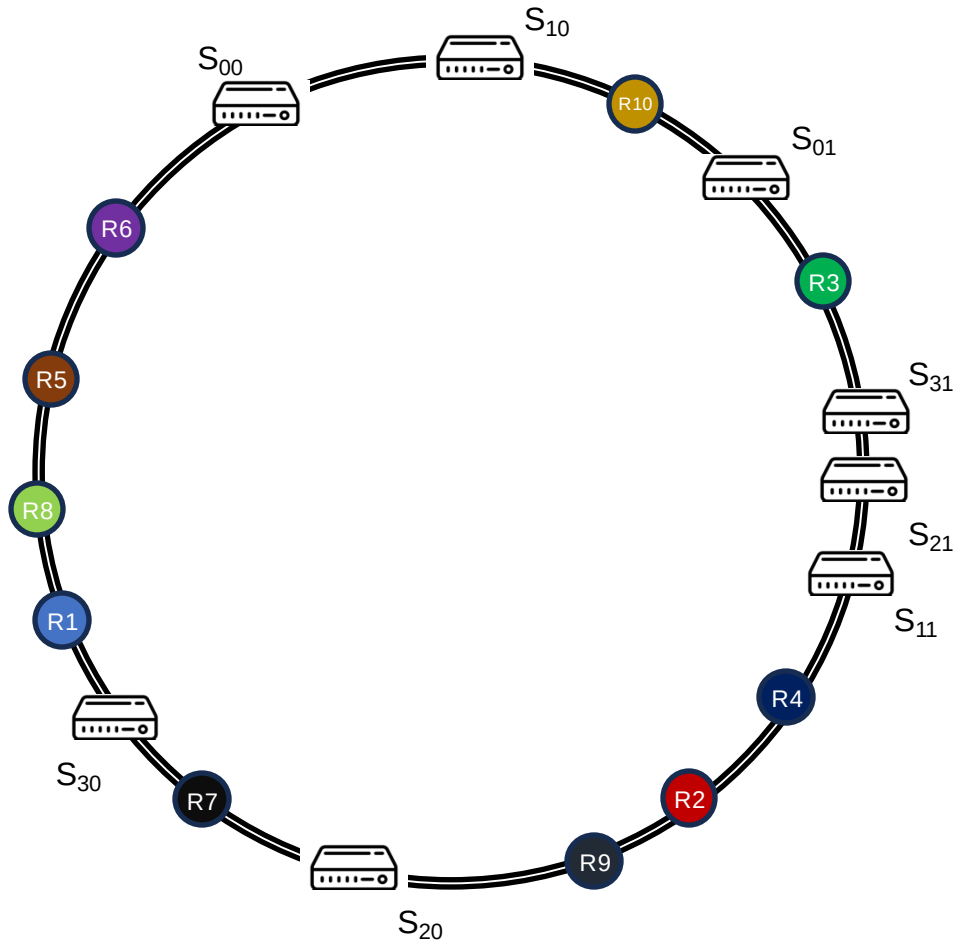
University of BRISTOL
Bristol Cyber Security Group

Create replicas of servers (virtual servers)

```
let hash = crypto.createHash('md5').update(`S${j}`).digest('hex');
```

- *hash*(*S00*) = fbe82363137ed7b697a5a51c84f20f55
- *hash*(*S01*) = 4cfe35d8825ba802868fabe687dc860d
- *hash*(*S10*) = 27d889be8a04837b5e6de309b746c57e
- *hash*(*S11*) = 7cc9261f6714c845c0bfadd2c13dd70e
- *hash*(*S20*) = a90da3af6adb3b66072942ca4e797958
- *hash*(*S21*) = 7c25eda1f2c0a2a5cbef09e6aa94d56f
- *hash*(*S30*) = c317634f2e2547e5909e87d80dd62c6e
- *hash*(*S31*) = 69bef73fcd68a2759a9dd451d2b04948

- *hash*(*R1*) = cda522d4353b166cc2dee84673307b4e
- *hash*(*R2*) = 8c6d22ff6f63fc6711cfa315cb80b314
- *hash*(*R3*) = 5c108ce0fe89d0632cfce75f650b36c2
- *hash*(*R4*) = 8717ce4dfdc86a4b576d9e983ab9fb29
- *hash*(*R5*) = e1b8054c9cdd622c9eeecfe71f26054a
- *hash*(*R6*) = f38ee76ce06ccd13940482aced1c7391
- *hash*(*R7*) = b9d7e51eba2c6d155e7fd3013338c38e
- *hash*(*R8*) = cfff813d86d447fa2a9c858650ebbb90
- *hash*(*R9*) = 8d28dad91e1ffada38203b9e5f9af86d
- *hash*(*R10*) = 33df7d5541bc0941336b1513815da7a5



Replica = 2

Consistent Hashing

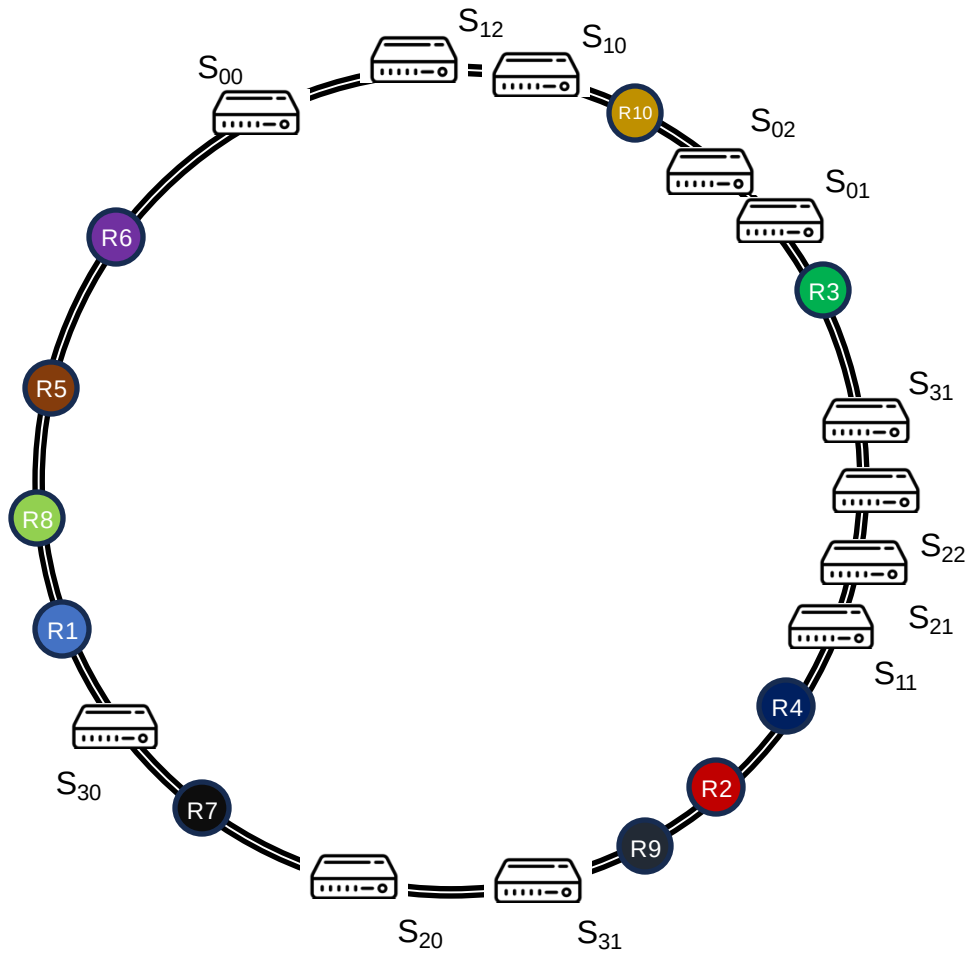
University of BRISTOL
Bristol Cyber Security Group

Create replicas of servers (virtual servers)

```
let hash = crypto.createHash('md5').update(`S${j}`).digest('hex');
```

- *hash*(*S00*) = fbe82363137ed7b697a5a51c84f20f55
- *hash*(*S01*) = 4cfe35d8825ba802868fabe687dc860d
- *hash*(*S02*) = 3aebec09aed29cb42464817e942cfb5b
- *hash*(*S10*) = 27d889be8a04837b5e6de309b746c57e
- *hash*(*S11*) = 7cc9261f6714c845c0bfadd2c13dd70e
- *hash*(*S12*) = 0dddeed4b0f51aaad8bcaf380fcbe48f
- *hash*(*S20*) = a90da3af6adb3b66072942ca4e797958
- *hash*(*S21*) = 7c25eda1f2c0a2a5cbef09e6aa94d56f
- *hash*(*S22*) = 75804b4c66c019655d1d2b56a2422fdd
- *hash*(*S30*) = c317634f2e2547e5909e87d80dd62c6e
- *hash*(*S31*) = 69bef73fcd68a2759a9dd451d2b04948
- *hash*(*S32*) = a1e28eee0339658d39a8b4d325b56e9c

- *hash*(*R1*) = cda522d4353b166cc2dee84673307b4e
- *hash*(*R2*) = 8c6d22ff6f63fc6711cfa315cb80b314
- *hash*(*R3*) = 5c108ce0fe89d0632cfce75f650b36c2
- *hash*(*R4*) = 8717ce4dfdc86a4b576d9e983ab9fb29
- *hash*(*R5*) = e1b8054c9cdd622c9eeecfe71f26054a
- *hash*(*R6*) = f38ee76ce06ccd13940482aced1c7391
- *hash*(*R7*) = b9d7e51eba2c6d155e7fd3013338c38e
- *hash*(*R8*) = cfff813d86d447fa2a9c858650ebbb90
- *hash*(*R9*) = 8d28dad91e1ffada38203b9e5f9af86d
- *hash*(*R10*) = 33df7d5541bc0941336b1513815da7a5



Replica = 3

Quiz

University of BRISTOL
Bristol Cyber Security Group

- Which of the following is a static load-balancing algorithm
  a) Round robin
  b) Least connections
  c) Consistent hashing

Quiz

University of BRISTOL
Bristol Cyber Security Group

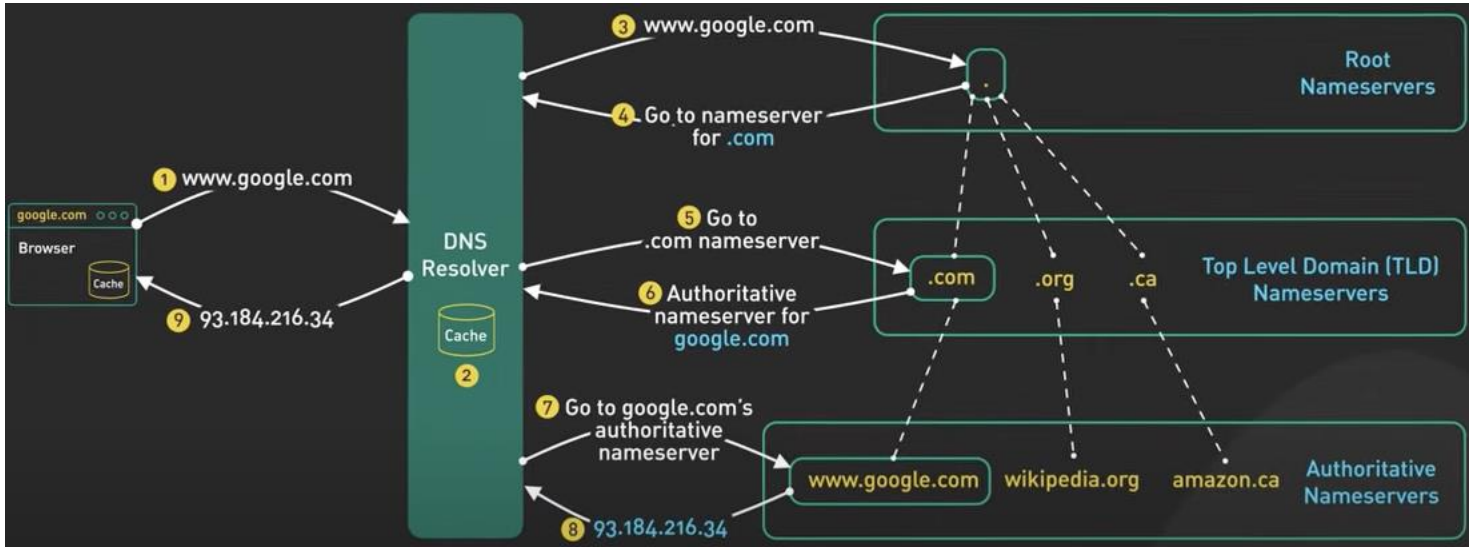
- What does session persistence in load balancing aim to achieve?

How do you get a request ID?

University of BRISTOL
Bristol Cyber Security Group

DNS Load Balancers

- The load balancer interacts with a Domain Name Services (DNS) infrastructure for a client name lookup. This returns a corresponding IP address which is then used as the ID
- Enables load balancing based on the geographic location of servers

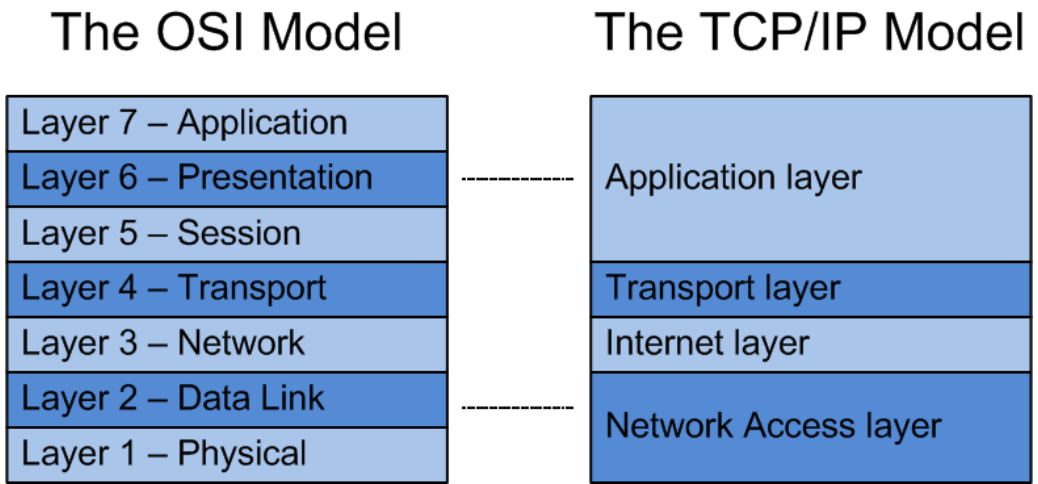


How do you get a request ID?

University of BRISTOL
Bristol Cyber Security Group

L4 Load Balancers

- Generates request ID based on information found in the network and transport layers of the request.
  - This includes the protocol (TCP, UDP, etc.), source and destination IP addresses, and source and destination port numbers.
  - The L4 LB doesn't have access to the contents of the request like application-level headers or the requested URL As a result, the routing decisions are based entirely on request headers at L4 and below.

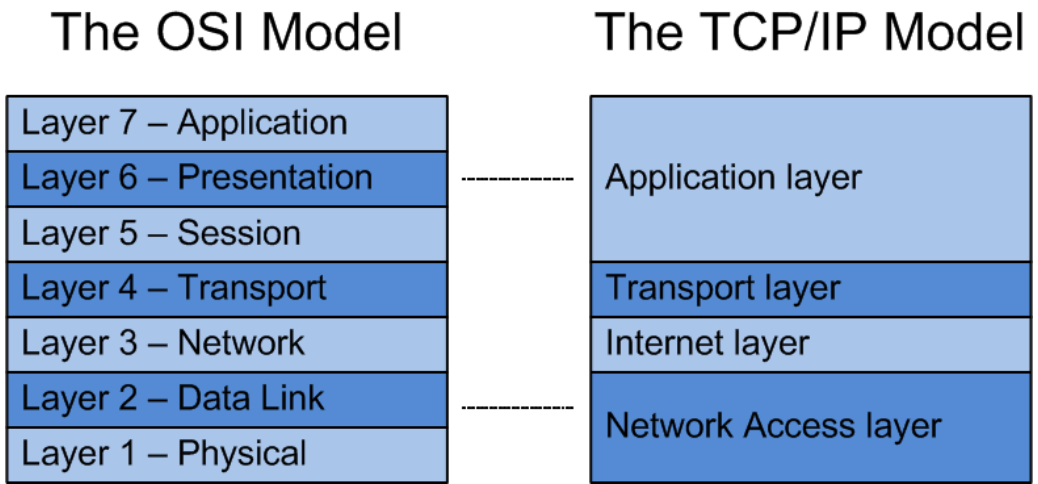


How do you get a request ID?

University of BRISTOL
Bristol Cyber Security Group

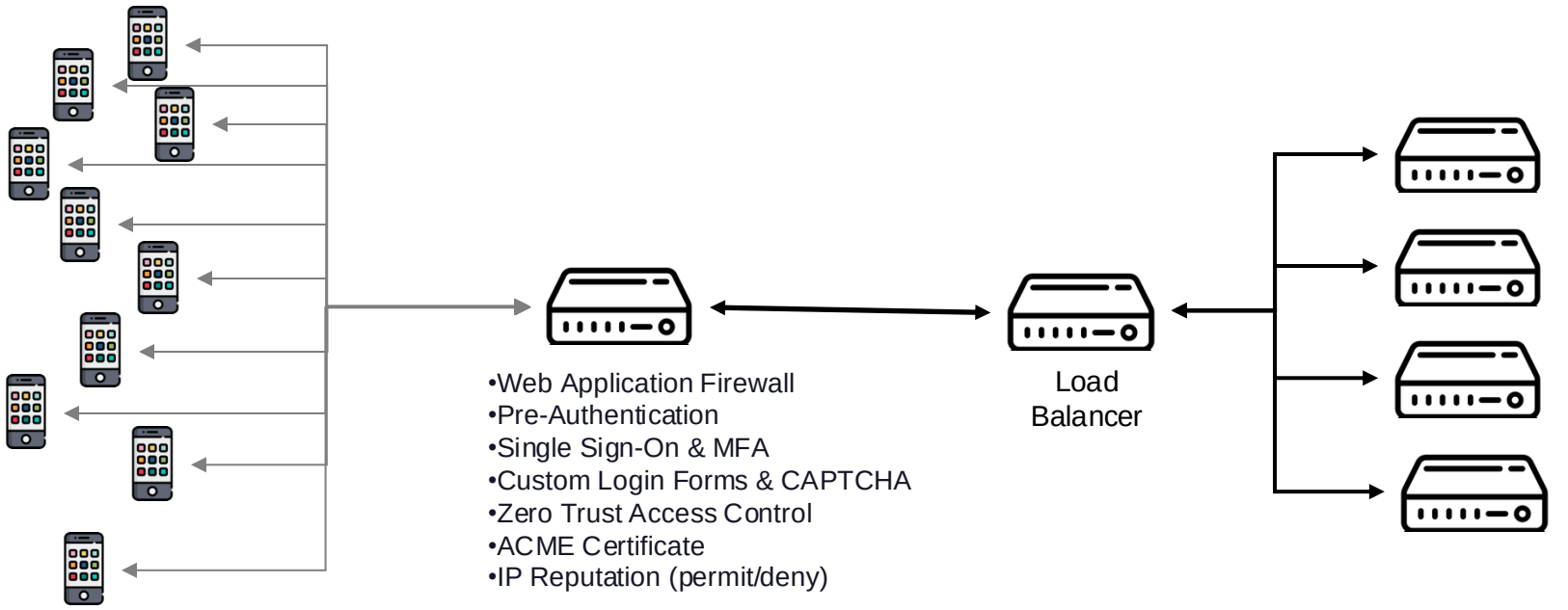
L7 Load Balancers

- Generates request ID based on information found in the application layer of the request.
  - Make more sophisticated routing decisions.
  - For session **stickiness**.



Load Balancers and Security

University of BRISTOL
Bristol Cyber Security Group



Load Balancers and Security

University of BRISTOL
Bristol Cyber Security Group

Link between LBs and servers

- L7 LBs need to examine the request to make routing decisions, any encrypted requests (HTTPS) need to be decrypted. That means an L7 LB will terminate the TCP connection with the request source and start a new connection to pass the request to the backend servers. This new request can be encrypted or not, depending how much you trust your backend server network.

- Load Balancers are also a natural point of protection against DDOS attacks. Since they generally prevent server overload by distributing requests well, in the case of a DDOS attack the LB makes it harder to overload the whole system. They also remove a single point of failure, and therefore make the system harder to attack.

Lab Workshop

University of BRISTOL
Bristol Cyber Security Group

https://github.com/Inah-Dev/LoadBalancerTestbed

Home Workshop

University of BRISTOL
Bristol Cyber Security Group

Create a Load Balancer on AWS

Practice creating a Load Balancer on AWS that forwards requests to a set of web servers running on EC2 instances.

Explore the robustness of your system design